

明白了，你再结把app侧和fw侧通信方式与调用链，各个静态库动态库之间的依赖关系在详细总结下，只需要faw项目，而且重点要保证com域和uds域内容的准确

- Thought for 41s
- 好的，下面以 FAW 项目 fawhq_e001_10 为例，做最终完整总结。

FAW 项目完整依赖与通信总结

一、进程与 .so 加载关系（FAW 分离模式，CONFIG_APP_GE_PROCESS=ON）

```
SOC 侧 (A核, Linux/QNX)

进程: dji_bf_app (DCOS_DOMAIN=0)
├── dlopen("com") → libfawhq_e001_10_com.so
│   ├── app_com_core_init() → COM通信核心入口
│   └── app_ge_core_init() → GE通用功能核心入口
└──

进程: dji_ad_app (DCOS_DOMAIN=0)
├── dlopen("app") → libfawhq_e001_10_app.so
│   └── app_ad_core_init() → AD自动驾驶核心入口
└──

进程: dji_doip_service_ipv4 (MINIDCOS_DOMAIN=1)
├── dlopen("uds_v4") → libuds_v4.so
│   └── app_uds_service_init() → UDS诊断v4核心入口
└──

进程: dji_doip_service_ipv6 (MINIDCOS_DOMAIN=1)
├── dlopen("uds_v6") → libfawhq_e001_10_uds_v6.so
│   └── app_uds_service_init() → UDS诊断v6核心入口
└──

(fawhq_e001_10 中 CONFIG_APP_UDSONIP_IPV6=OFF, 实际不编译)

进程: dji_log_export
(独立可执行文件, 不加载 .so)
```

- 关键区分：
- COM 和 GE 不是两个独立进程，而是编译进同一个 lib*_com.so，由 dji_bf_app 一个进程统一加载
 - UDS 是独立进程 + 独立 .so，且运行在 MINIDCOS_DOMAIN(1)，与 COM/AD 不在同一个 DCOS 域
 - UDS v4 的 .so 命名特殊：是 libuds_v4.so（无产品前缀），因为 check_autosar_uds_v4_build_first_run 机制确保多产品编译时只生成一份

二、每个 .so 的静态库组成（--whole-archive 打包）

2.1 libfawhq_e001_10_com.so —— COM域 + GE域

由 consys_bf.cmake 的 product_build_marcos 宏构建。
CONFIG 开关依据：fawhq_e001_10_config.cmake + faw_config.cmake

CONFIG 宏	值	作用
CONFIG_APP_COM_CORE	ON	编译 COM 通信核心
CONFIG_APP_GE_CORE	ON	编译 GE 通用功能核心
CONFIG_SIP_AUTOSAR_COM	ON	链接 AUTOSAR COM 协议栈配置
CONFIG_X_DOM_CAN_RT_COM	ON	链接 CAN 跨域路由(COM侧)
CONFIG_X_DOM_SOMEIP_RT_COM	ON	链接 SOME/IP 跨域路由(COM侧)
CONFIG_BF_OEM_FEATURE	ON	链接 BF OEM特性
CONFIG_OEM_FEATURE	ON	链接 OEM特性
CONFIG_OEM_PRODUCT_FEATURE	ON	链接 OEM产品特性

```
libfawhq_e001_10_com.so
├──
├── [--whole-archive 静态打包]
├──
├── plat_bf_cdd → dsar-plat-bf conan 包
├── dcoss_dcms → middleware conan 包
├──
├── app_com_core_fawhq_e001_10 → src/dsar_app/app_core/app_com_core/
├── 源码: app_com_core.cpp (单个文件, COM核心入口+话题注册)
```


由 consys_ad.cmake 的 app_product_build_marcos 宏构建。

CONFIG 宏	值	作用
CONFIG_APP_AD_CORE	ON	链接 AD 核心框架
CONFIG_X_DOM_CAN_RT_AD	ON	链接 CAN 跨域路由(AD侧)
CONFIG_X_DOM_SOMEIP_RT_AD	ON	链接 SOME/IP 跨域路由(AD侧)
CONFIG_OEM_FEATURE	ON	链接 OEM特性(AD侧)
CONFIG_OEM_FEATURE_IA	ON	链接 IA OEM特性
CONFIG_IPSIGNAL_SAIL2SOC	(取决于平台)	链接 IPSignal 代理

```
libfawhq_e001_10_app.so
├── [--whole-archive 静态打包]
├──
├── plat_bf_cdd                - dsar-plat-bf conan 包
├── mini_dcos                  - middleware conan 包
├── dsar_plat_config            - dsar-plat conan 包
├──
├── vehicle_proxy_fawhq_e001_10 - 车辆信号代理
├── 源码: product/faw/fawhq_e001_10/proxy/
├── 子目录: internal/, comif_cfg/, SensorGroup/
├──
├── adapter_fawhq_e001_10      - 信号适配器
├──   ├── common_adapter (通用适配)
├──   ├── parking_adapter (泊车适配)
├──   ├── driving_adapter (行车适配)
├──   ├── as_adapter (辅助驾驶适配)
├──   └── hmiapp_adapter (HMI应用适配, 含OEM版本)
├──
├── product_param              - PAL参数编译产物
├── product_fusa_*              - 功能安全
├── product_runnable_fawhq_e001_10 - 可运行实体
├──
├── plat_ad_app_core            - dsar-plat-ad conan 包 (AD核心框架)
├── plat_ad_demo                - dsar-plat-ad conan 包
├── plat_ad_fusa_app            - dsar-plat-ad conan 包
├── plat_vehicle_proxy          - dsar-plat-ad conan 包
├── plat_common                 - dsar-plat-ad conan 包
├── plat_ad_tlv                 - dsar-plat-ad conan 包
├── plat_ad_cndtm               - dsar-plat-ad conan 包
├──
├── app_ia_plat                 - IA(智能助手)平台层
├── qpc                          - QPC框架(状态机)
├──
├── oem_feature_app_fawhq_e001_10 - OEM特性(AD侧)
├── x_dom_can_rt_gen_lib_app_fawhq_e001_10 - CAN信号路由(AD侧)
├── x_dom_someip_rt_gen_lib_app_fawhq_e001_10 - SOME/IP路由(AD侧)
├──
├── gateway_hmi_com             - 网关HMI通信
├── adc_protobuf                - ADC protobuf
├── dji_gateway_proto-dependency - DJI网关协议依赖
├──
├── [动态链接]
├── gateway                     - conan 外部库
├── znet                         - conan 外部库
├── dji_stream_media_protocol   - conan 外部库
```

三、.so 之间的依赖关系

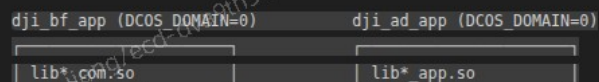
结论：4 个 .so 之间编译时完全互不依赖。证据：

- 每个 .so 的 target_link_libraries 中不包含任何其他 .so
- 每个 .so 设置了 -Wl,--no-undefined，如果依赖未解析符号会链接失败
- 所有 .so 都通过 --whole-archive 自包含所需的静态库

但运行时它们共享了同一套 conan 平台包的不同副本（多次链接），比如 plat_bf_cdd 在 lib*_com.so、libuds_v4.so、lib*_app.so 中各有一份。

四、跨域/跨进程通信机制

4.1 SOC 侧跨进程通信：全部走 DCMS



依赖	来源	使用方
plat_bf_cdd	dsar-plat-bf conan 包	所有 .so + 所有可执行文件
plat_bf_ge 及其子模块	dsar-plat-bf conan 包	lib* com.so (GE域)

plat_bf_udsonip	dsar-plat-bf conan 包	libuds_v4.so
plat_bf_log_export	dsar-plat-bf conan 包	dji_log_export
plat_ad_*	dsar-plat-ad conan 包	lib*_app.so (AD域)
dsar_plat_config	dsar-plat conan 包	COM域 + AD域 + 可执行文件
dcos_dcms, mini_dcos	middleware conan 包	所有 .so
xml2	conan	所有可执行文件
gateway, znet, dji_stream_media_protocol	conan	lib*_app.so
AUTOSAR BSW 源码	dsar-sip git submodule	COM域(COM栈) + UDS域(UDS栈)
x_dom_can_rt SOC端源码	dsar-plat-bf conan 包	COM域 + AD域
x_dom_can_rt MCU端源码	dsar-sip git submodule	FW侧
x_dom_someip_rt 源码	SIP生成, 产品目录中	COM域 + UDS域 + AD域 + FW侧
dsar-bsw	dsar-bsw conan 包	FW侧 (BSP/RTOS/MCAL)
dji-message	dji-message conan 包	FW侧 + APP侧

七、关键纠正：COM域 与 UDS域 的准确关系

之前统称 COM+UDS 为 “BF域的两个子域”，这个说法在概念层面正确，但在物理部署层面不准确。准确的说法是：

	COM域	GE域	UDS域
源码位置	app_core/app_com_core/	app_core/app_ge_core/	app_core/app_udsonip/
编译产物	静态库 → 打包进 lib*_com.so	静态库 → 打包进 lib*_com.so	静态库 → 打包进 libuds_v4.so
运行进程	dji_bf_app	dji_bf_app	dji_doip_service_ipv4
DCOS 域	DCOS_DOMAIN(0)	DCOS_DOMAIN(0)	MINIDCOS_DOMAIN(1)
共享的平台层	plat_bf_cdd + dcos_dcms	mini_dcos + plat_bf_ge	plat_bf_cdd + plat_bf_udsonip

COM 和 GE 在物理上是同一个 .so、同一个进程，只是源码层面分了两个 app_core 子目录、两个 _core_init 入口。UDS 则是完全独立的进程和 .so，连 DCOS 域都不同。